Experiment 4: (stack and queue)

Q1) Write a program using C to create a stack of numbers and perform using UDF:

(i) push operation (ii) pop operation (iii) display operation

```
#include <stdio.h>
#define MAX 100

int stack[MAX], top=-1;

void push(int x){
   if(top==MAX-1) return;
   stack[++top]=x;
}

int pop(){
   if(top==-1) return -1;
   return stack[top--];
}

void display(){
   for(int i=top;i>=0;i--) printf("%d ",stack[i]);
}

int main(){
   int ch,x;
   do{
      scanf("%d",&ch);
      if(ch==1){ scanf("%d",&x); push(x); }
      if(ch==2){ pop(); }
      if(ch==3){ display(); }
   }while(ch!=4);
   return 0;
}
```

Q2) Write a C program to create a linear queue and perform the following operations using UDF:

(i) insertion ii) deletion and iii) Traversal

```
#include <stdio.h>
#define MAX 100
int queue[MAX], front=0, rear=0;

void insert(int x){
   if(rear==MAX) return;
   queue[rear++]=x;
}
int delete(){
   if(front==rear) return -1;
   return queue[front++];
}
void traverse(){
   for(int i=front;i<rear;i++) printf("%d ",queue[i]);
}
int main(){
   int ch,x;
   while(1){
      scanf("%d",&ch);
      if(ch==1){ scanf("%d",&x); insert(x); }
      else if(ch==2){ delete(); }
      else if(ch==3){ traverse(); }
      else break;
   }
   return 0;
}
```

Q3) Write a C program to create a circular queue and perform the following operations using UDF:

(i) insertion ii) deletion and iii) Traversal

```
#include <stdio.h>
#define MAX 5
int cq[MAX], front=-1, rear=-1;
void insert(int x){
   if((rear+1)%MAX==front) return;
   if(front==-1) front=rear=0;
   else rear=(rear+1)%MAX;
   cq[rear]=x;
}
int delete(){
   if(front==-1) return -1;
   int val=cq[front];
   if(front==rear) front=rear=-1;
   else front=(front+1)%MAX;
   return val;
}
void traverse(){
   if(front==-1) return;
   int i=front;
   while(1){
      printf("%d ",cq[i]);
      if(i==rear) break;
      i=(i+1)%MAX;
   }
}
int main(){
   int ch,x;
   while(1){
      scanf("%d",&ch);
      if(ch==1){ scanf("%d",&x); insert(x); }
      else if(ch==2){ delete(); }
      else if(ch==3){ traverse(); }
      else break;
   }
   return 0;
}
```

Experiment 5: (searching and sorting)

Q1) Write a program to implement binary search on array elements using UDF

```
#include <stdio.h>


int binary(int a[],int n,int key){
   int l=0,r=n-1,mid;
   while(l<=r){
      mid=(l+r)/2;
      if(a[mid]==key) return mid;
      else if(a[mid]<key) l=mid+1;
      else r=mid-1;
   }
   return -1;
}

int main(){
   int n,a[100],key;
   scanf("%d",&n);
   for(int i=0;i<n;i++) scanf("%d",&a[i]);
   scanf("%d",&key);
   binary(a,n,key);
   return 0;
}
```

Q2) Write a program to implement selection sort on a given list of array elements.

```
#include <stdio.h>

void selection(int a[],int n){
   int i,j,min,temp;
   for(i=0;i<n-1;i++){
      min=i;
      for(j=i+1;j<n;j++)
         if(a[j]<a[min]) min=j;
      temp=a[i]; a[i]=a[min]; a[min]=temp;
   }
}

int main(){
   int n,a[100];
   scanf("%d",&n);
   for(int i=0;i<n;i++) scanf("%d",&a[i]);
   selection(a,n);
   return 0;
}
```

Q3) Write a program to input a string and sort the alphabets in ascending order using bubble sort.

```
#include <stdio.h>
#include <string.h>

void bubble(char s[]){
   int n=strlen(s),i,j;
   char temp;
   for(i=0;i<n-1;i++)
      for(j=0;j<n-i-1;j++)
         if(s[j]>s[j+1]){
            temp=s[j];
            s[j]=s[j+1];
            s[j+1]=temp;
         }
}

int main(){
   char s[100];
   scanf("%s",s);
   bubble(s);
   return 0;
}
```

Experiment 6: (sorting and merging)

Q1) Write a program to input elements into two arrays A[5] and B[5]. Input the elements in ascending order and then merge their values into a resultant array C[10] in sorted manner using UDF.

```
#include <stdio.h>


void merge(int A[],int B[],int C[]){
   int i=0,j=0,k=0;
   while(i<5 && j<5){
      if(A[i]<B[j]) C[k++]=A[i++];
      else C[k++]=B[j++];
   }
   while(i<5) C[k++]=A[i++];
   while(j<5) C[k++]=B[j++];
}


int main(){
   int A[5],B[5],C[10];
   for(int i=0;i<5;i++) scanf("%d",&A[i]);
   for(int i=0;i<5;i++) scanf("%d",&B[i]);
   merge(A,B,C);
   return 0;
}
```

Q2) Write a program to implement insertion sort on a given list of array elements.

```
#include <stdio.h>
void insertion(int a[],int n){
   int i,j,key;
   for(i=1;i<n;i++){
      key=a[i];
      j=i-1;
      while(j>=0 && a[j]>key){
         a[j+1]=a[j];
         j--;
      }
      a[j+1]=key;
   }
}
int main(){
   int n,a[100];
   scanf("%d",&n);
   for(int i=0;i<n;i++) scanf("%d",&a[i]);
   insertion(a,n);
   return 0;
}
```

Q3) Write a C program to implement quick sort to a given list of integers to sort in ascending order.

```
#include <stdio.h>
int partition(int a[],int low,int high){
   int pivot=a[high];
   int i=low-1,temp;
   for(int j=low;j<high;j++){
      if(a[j]<pivot){
         i++;
         temp=a[i]; a[i]=a[j]; a[j]=temp;
      }
   }
   temp=a[i+1]; a[i+1]=a[high]; a[high]=temp;
   return i+1;
}
void quicksort(int a[],int low,int high){
   if(low<high){
      int p=partition(a,low,high);
      quicksort(a,low,p-1);
      quicksort(a,p+1,high);
   }
}
int main(){
   int n,a[100];
   scanf("%d",&n);
   for(int i=0;i<n;i++) scanf("%d",&a[i]);
   quicksort(a,0,n-1);
   return 0;
}
```